

AI CEO

AUTONOMOUS ENTERPRISE INTELLIGENCE PLATFORM

LAYER 1 — CONNECTOR & UNIFIED DATA INGESTION

Master Build Prompt for an AI Coding Agent

Purpose: Give another AI everything required to design, implement, test, run, and verify a fully functioning first layer of the AI CEO platform.

Item	Specification
Project	AI CEO — Autonomous Enterprise Intelligence Platform
Layer	Layer 1: Connector & Unified Data Ingestion
Primary role	Read-only ingestion of enterprise-style data into a canonical schema
Initial sources	Odoo/demo enterprise data + CSV/Excel; adapter architecture must support future REST/GraphQL systems
Core backend	Python + FastAPI
Persistence	PostgreSQL
Data processing	pandas / Pydantic / SQLAlchemy or SQLModel
Deployment	Docker Compose for local reproducible execution
Boundary	No Neo4j, RAG, forecasting, agents, or write-back actions in Layer 1
Success condition	A clean source-to-canonical-data pipeline that can be run end-to-end and consumed by Layer 2

SOURCE-OF-TRUTH ALIGNMENT

The project proposal defines the Connector Layer as read-only API connectors and CSV ingestion that normalize data from each source system into a common schema. The first implementation phase is explicitly the connector framework, with the deliverable being read-only demo CRM/ERP ingestion and completed schema mapping.

This prompt intentionally expands that requirement into an engineering-grade implementation specification while preserving the project's prototype boundary: representative enterprise systems, read-only access, and no production write access.

Version: 1.0 • Intended for direct use with a coding AI

1. HOW THIS DOCUMENT MUST BE USED

Copy the complete MASTER BUILD PROMPT in Section 2 into the coding AI. The coding AI should treat it as an implementation contract, not as a brainstorming request. It must inspect the repository first, preserve useful existing work, then implement Layer 1 incrementally until every acceptance criterion passes.

- **Do not stop at architecture.** The expected output is runnable source code, migrations, seed/demo data, tests, Docker configuration, API documentation, and a verified end-to-end pipeline.
- **Do not fake integrations.** If a real external API cannot be safely used, implement a deterministic local adapter/demo connector with the same interface and document the boundary.
- **Keep Layer 1 isolated.** Build interfaces that make Layer 2 easy, but do not implement the graph, vector database, RAG, forecasting, or multi-agent system now.
- **Make every result reproducible.** A fresh machine should be able to start the stack, load demo data, run ingestion, query normalized records, and execute tests.

Recommended execution sequence

Step	Coding AI action	Required output
1	Inspect existing repository and determine what already exists	Repository audit + implementation plan
2	Create/normalize project structure	Clean Layer 1 modules
3	Implement canonical models and PostgreSQL schema	Migrations + ORM models
4	Implement source adapters	CSV/Excel + Odoosdemo + generic REST adapter
5	Implement normalization and validation	Deterministic transformation pipeline
6	Implement ingestion orchestration	Idempotent sync jobs + run tracking
7	Expose operational APIs	FastAPI endpoints + OpenAPI docs
8	Add tests and observability	Unit/integration/API tests + logs/metrics
9	Containerize and verify	Docker Compose + health checks
10	Run final acceptance suite	Evidence that Layer 1 is fully functional

2. MASTER BUILD PROMPT

The following section is the prompt to give directly to the implementation AI.

ROLE

You are the lead software architect and senior Python backend engineer responsible for building **Layer 1** of the AI CEO — Autonomous Enterprise Intelligence Platform. You are not being asked to produce a mockup, pseudoproject, or partial scaffold. You must build a working, testable, locally deployable implementation.

You must reason from first principles, make the architecture explicit, implement the code, run the tests, fix failures, and leave the repository in a state where a human developer can start the system and demonstrate Layer 1 without manually repairing the code.

PROJECT CONTEXT

AI CEO is a multi-agent decision layer above a mid-sized company's existing systems of record. The full architecture is: **CRM/ERP/HRMS Systems → Connector Layer → Enterprise Twin + Knowledge Graph → Multi-Agent AI → Human Sign-off + Action.** Layer 1 is the Connector Layer. Its job is to safely extract enterprise-style data and transform it into a stable canonical representation that later layers can consume.

The project proposal specifically scopes Layer 1 to read-only API connectors and CSV/Excel ingestion, with normalization into a common schema. The prototype may use a demo enterprise system such as Odoo or a public/simulated enterprise-style dataset. Production write access is out of scope.

NON-NEGOTIABLE RULES

- **Read-only only.** Never create, update, delete, or mutate records in a source CRM/ERP/HRMS/support system.
- **No secrets in code.** Credentials must come from environment variables or a secure configuration mechanism.
- **No silent data corruption.** Invalid records must be rejected, quarantined, or explicitly reported; never silently dropped.
- **Idempotency.** Re-running the same source file or API page must not create duplicate canonical records.
- **Provenance.** Every canonical record must retain enough metadata to identify its source system, source entity, source record ID, ingestion run, and timestamps.
- **Deterministic normalization.** The same source record and mapping configuration should produce the same canonical representation.
- **Future compatibility.** Layer 2 should be able to consume canonical entities without knowing which source produced them.
- **Do not implement later layers now.** Neo4j, vector search, RAG, XGBoost/TFT, agents, Copilot, and workflow execution are interfaces/future consumers only.
- **Demonstrable.** Include realistic demo data and a repeatable end-to-end demonstration.

3. LAYER 1 OBJECTIVES AND BOUNDARIES

Primary objective

Build a production-style prototype ingestion subsystem that can read multiple enterprise data sources through a common adapter interface, validate and normalize their records, persist canonical entities in PostgreSQL, track ingestion runs, and expose the resulting data through stable APIs.

What Layer 1 must answer

- Can the system connect to an enterprise-style source without writing to it?
- Can heterogeneous source fields be mapped into one canonical schema?
- Can duplicate, malformed, missing, and inconsistent records be handled predictably?
- Can the same dataset be ingested repeatedly without duplication?
- Can we tell exactly where every canonical record came from?
- Can a downstream Knowledge Graph or analytics service consume the normalized records without knowing the source-specific schema?

Explicitly OUT OF SCOPE

Not part of Layer 1	How it is handled now
Neo4j enterprise knowledge graph	Define canonical entities and relationships as downstream-ready contracts only.
Vector DB / embeddings / RAG	Do not implement. Preserve document/text fields and metadata needed later.
Churn prediction	Do not train or serve the model in Layer 1.
Revenue/demand forecasting	Do not train or serve the model in Layer 1.
14 functional agents + CEO Agent	Do not implement.
LLM reasoning	Do not use an LLM for deterministic ingestion/normalization.
Source write-back	Forbidden.
Production enterprise integrations	Use adapters/interfaces and demo-safe connectors.
Frontend dashboard	Optional minimal API docs/admin visibility only; do not let UI work block backend completion.

Definition of done

Layer 1 is complete only when a clean environment can start PostgreSQL and the API, load demo source data, execute an ingestion run, normalize records into canonical tables, record the run outcome and errors, expose canonical records through documented APIs, and pass the complete automated test suite.

4. TARGET ARCHITECTURE



Design principle

Source-specific complexity must terminate at the connector + normalization boundary. Everything after that boundary should work with canonical entities and provenance metadata.

Recommended module responsibilities

Module	Responsibility
connectors/	Source adapters; no business reasoning.
schemas/source/	Source-specific DTOs where useful.
schemas/canonical/	Stable canonical Pydantic contracts.
normalization/	Mapping, coercion, cleaning, relationship resolution.

Module	Responsibility
validation/	Schema and business-quality checks.
ingestion/	Run orchestration, batching, idempotency, checkpoints.
persistence/	ORM models, repositories, transactions, migrations.
api/	FastAPI routes for health, sources, runs, and canonical data.
observability/	Structured logging, counters, timing, error reporting.
tests/	Unit, integration, contract, and end-to-end tests.
scripts/	Seed data and reproducible demo commands.

5. SOURCE CONNECTOR SPECIFICATION

Implement the connector layer around an explicit interface. The application must not contain source-specific code scattered throughout services.

Canonical connector interface

```
class SourceConnector(Protocol):
    source_name: str
    source_type: str

    def health_check(self) -> ConnectorHealth: ...

    def list_entities(self) -> list[SourceEntity]: ...

    def fetch_entities(
        self,
        entity_type: str,
        cursor: str | None = None,
        page_size: int = 100,
    ) -> Page[dict]: ...

    def get_entity(
        self,
        entity_type: str,
        source_id: str,
    ) -> dict: ...

    def capabilities(self) -> ConnectorCapabilities: ...
```

Connector 1 — CSV/Excel

- Support CSV as mandatory; Excel may be supported through pandas/openpyxl.
- Accept a directory or configured file set containing entities such as customers, employees, deals, projects, tickets.
- Support configurable column mapping rather than hard-coding one filename.
- Record filename, file hash, row number, ingestion timestamp, and ingestion_run_id.
- Use deterministic row identity: source file identity + configured business key or stable row hash.
- Handle encoding issues, empty rows, duplicate rows, missing headers, malformed dates, and invalid numeric values.
- Never overwrite the original raw value in the raw ingestion boundary.

Connector 2 — Odoo/demo ERP-CRM

- Implement a safe read-only Odoo adapter for the chosen demo environment or provide a local Odoo-compatible mock adapter if credentials/network access are unavailable.
- Typical representative entities: customers/partners, employees, sales orders/deals, projects, support tickets where available.
- Isolate Odoo authentication and RPC/HTTP details inside the adapter.
- Support pagination/batching and retries.
- Do not expose Odoo-specific field names beyond the adapter/mapper boundary.

Connector 3 — Generic REST

- Implement a reusable REST adapter contract with configurable base URL, auth mode, endpoint, pagination strategy, and JSON path.
- Support GET only.
- Support bearer/API-key authentication via environment variables.
- Support timeout, retry with exponential backoff, response validation, and rate-limit handling.

GraphQL readiness

The proposal permits REST/GraphQL connectors. The first working implementation may prioritize REST and Odooc/CSV, but the adapter abstraction must not prevent a GraphQL connector from being added without changing canonical models or ingestion orchestration.

6. CANONICAL ENTERPRISE SCHEMA

The canonical schema is the most important contract in Layer 1. It must be source-neutral, stable, typed, documented, and explicitly designed for later Knowledge Graph, RAG, forecasting, and agentic consumers.

Every entity should share these provenance fields

Field	Type	Requirement
id	UUID	Internal canonical primary key.
source_system	string	Logical source name, e.g. odoo_demo or csv_demo.
source_entity	string	Original entity/table/resource type.
source_id	string	Stable source identifier.
source_updated_at	datetime nullable	Timestamp from source when available.
ingested_at	datetime	When this version entered Layer 1.
ingestion_run_id	UUID	Run that produced/updated the record.
record_hash	string	Deterministic hash of normalized business content.
is_active	boolean	Logical active state where meaningful.

Core entities

Entity	Minimum fields	Downstream value
Organization	name, industry, country, status	Enterprise boundary / tenant.
Employee	name, email, department, title, manager_source_id, status, hire_date	HR and ownership relationships.
Customer	name, email, segment, industry, status, owner_source_id, created_at	Customer/account intelligence.
Deal	name, customer_source_id, owner_source_id, stage, amount, currency, probability, expected_close_date	Sales/revenue intelligence.
Project	name, customer_source_id, owner_source_id, status, start_date, end_date, budget	Delivery/resource context.
SupportTicket	customer_source_id, priority, status, category, subject, description, created_at, resolved_at, assignee_source_id	Risk/support signals.
Document	title, document_type, body_text, source_uri, owner_source_id, created_at, updated_at	Future RAG/institutional memory.

Relationship keys

Where possible, retain both the source foreign key and a canonical resolved key. Example: Deal.customer_source_id identifies the source customer while customer_id points to the canonical Customer row. If a relationship cannot be resolved, preserve the unresolved source key and emit a validation warning/error rather than silently dropping the relationship.

7. DATA CONTRACTS AND NORMALIZATION RULES

Type normalization

Input variation	Canonical treatment
1,000 / 1.000 / 1000	Normalize according to configured locale/source rules; never guess silently.
■ 50,000 / INR 50000	Store numeric amount + ISO currency code separately.
DD/MM/YYYY vs YYYY-MM-DD	Parse using source-specific format configuration; reject ambiguous values.
true / True / yes / 1	Normalize to boolean using explicit accepted values.
NULL / N/A / - / empty	Map configured null tokens to null.
Emails with surrounding spaces	Trim and lowercase for normalized comparison; preserve raw input separately.
Phone numbers	Normalize conservatively; preserve original value.
Status labels	Map source enum to canonical enum through configuration; unknown values become warnings/errors.

Identifier and deduplication strategy

- Primary identity: (source_system, source_entity, source_id).
- Secondary content identity: record_hash of canonical business fields.
- Never use name alone as a primary identity.
- For file rows without a stable ID, require a configured business key or deterministic row hash and document its limitations.
- If two source records appear to represent the same real-world entity but have different source IDs, do not merge them automatically in Layer 1. Preserve both and expose a future entity-resolution hook.

Versioning strategy

Use upsert semantics keyed by source identity. Maintain either current-state tables plus ingestion history, or current-state tables plus an immutable source_records table. The recommended prototype design is current canonical tables + source_records + ingestion_runs, so changes can be audited without making the application unnecessarily complex.

Raw vs canonical data

- **Raw boundary:** preserves source payload/row for audit and debugging.
- **Canonical boundary:** contains validated normalized values used by downstream layers.
- **Never make downstream consumers parse raw source-specific payloads.**

8. POSTGRESQL DATA MODEL

Create proper migrations. Do not rely on an application startup script that mutates the schema invisibly.

Required operational tables

Table	Purpose
ingestion_runs	One row per execution, with source, status, timing, counts, and error summary.
ingestion_errors	Structured rejected-record and connector errors.
source_records	Raw/source representation plus provenance and content hash.
connector_configs	Non-secret connector configuration; secrets referenced by environment variable names.

Required canonical tables

organizations, employees, customers, deals, projects, support_tickets, and documents.

Recommended indexes

- (source_system, source_id, source_entity) unique where applicable.
- source_updated_at for incremental sync.
- ingestion_run_id for audit queries.
- email where frequently queried, with appropriate normalization.
- foreign keys for canonical relationships.
- created_at / updated_at for operational queries.

Transaction policy

- Process records in bounded batches rather than one giant transaction.
- A malformed individual record should not corrupt an entire successful batch.
- The run must finish with a truthful status: SUCCESS, PARTIAL_SUCCESS, FAILED, or NOOP.
- If a batch transaction fails, rollback that batch and record the reason.

9. END-TO-END INGESTION WORKFLOW

1. Create ingestion_run
2. Load connector configuration
3. Authenticate / health-check source
4. Discover supported entity types
5. Fetch a page/batch
6. Persist raw source payload with provenance
7. Map source fields to canonical fields
8. Normalize values
9. Validate canonical object
10. Resolve canonical foreign keys where possible
11. Upsert canonical record
12. Record warnings/errors
13. Commit batch
14. Advance checkpoint/cursor
15. Continue until source is exhausted
16. Finalize ingestion_run statistics
17. Expose run summary through API

Incremental ingestion

- Implement a connector capability indicating whether incremental sync is supported.
- Prefer source_updated_at / modified_at filters where available.
- Persist the last successful cursor or watermark per source/entity.
- Never advance the watermark past a failed batch.
- Provide a full-refresh mode for deterministic demos and recovery.

Idempotency test

Running the exact same ingestion command twice must result in the same number of canonical records after the second run, with no duplicate source identities. The second run should be reported as NOOP or as an update with zero semantic changes, depending on implementation.

Failure recovery

- Transient network errors: retry with bounded exponential backoff.
- Authentication failure: fail fast and report actionable error.
- Malformed source record: quarantine and continue where safe.
- Database outage: fail the run and preserve enough metadata to retry.
- Rate limiting: respect Retry-After when available and cap concurrency.

10. FASTAPI API CONTRACT

The API must be versioned from the beginning. Use /api/v1.

Endpoint	Method	Purpose
/api/v1/health	GET	Liveness/readiness status.
/api/v1/sources	GET	List configured source connectors and capabilities.
/api/v1/sources/{source}/health	GET	Run connector health check.
/api/v1/ingestion/runs	POST	Start an ingestion run.
/api/v1/ingestion/runs	GET	List runs with filters.
/api/v1/ingestion/runs/{run_id}	GET	Run detail and counts.
/api/v1/ingestion/runs/{run_id}/errors	GET	Structured ingestion errors.
/api/v1/entities/{entity_type}	GET	Query canonical entities with pagination.
/api/v1/entities/{entity_type}/{id}	GET	Fetch one canonical entity.
/api/v1/metrics/ingestion	GET	Operational ingestion metrics.

API requirements

- OpenAPI schema must be automatically generated and accurate.
- Use Pydantic request/response models; do not return ORM objects directly.
- Pagination must be explicit and stable.
- Return structured errors with code, message, details, and request/run correlation ID.
- Do not expose connector secrets.
- Do not expose raw payloads through public APIs unless an explicit debug/admin route is protected and documented.

Example ingestion request

```
POST /api/v1/ingestion/runs

{
  "source": "csv_demo",
  "entities": ["customers", "employees", "deals", "projects", "support_tickets"],
  "mode": "full",
  "dry_run": false
}
```

11. REQUIRED REPOSITORY STRUCTURE

```
ai-ceo/
├── app/
│   ├── main.py
│   ├── core/
│   │   ├── config.py
│   │   ├── logging.py
│   │   └── security.py
│   ├── api/
│   │   ├── v1/
│   │   │   ├── router.py
│   │   │   ├── health.py
│   │   │   ├── sources.py
│   │   │   ├── ingestion.py
│   │   │   └── entities.py
│   │   ├── connectors/
│   │   │   ├── base.py
│   │   │   ├── csv_connector.py
│   │   │   ├── odoo_connector.py
│   │   │   └── rest_connector.py
│   │   ├── schemas/
│   │   │   ├── source/
│   │   │   ├── canonical/
│   │   │   └── normalization/
│   │   │       ├── mappings.py
│   │   │       ├── coercion.py
│   │   │       ├── identifiers.py
│   │   │       └── pipeline.py
│   │   ├── validation/
│   │   │   ├── rules.py
│   │   │   └── quarantine.py
│   │   ├── ingestion/
│   │   │   ├── orchestrator.py
│   │   │   ├── checkpoint.py
│   │   │   └── run_service.py
│   │   ├── persistence/
│   │   │   ├── models/
│   │   │   ├── repositories/
│   │   │   ├── database.py
│   │   │   └── unit_of_work.py
│   │   ├── observability/
│   │   │   ├── metrics.py
│   │   │   └── tracing.py
│   ├── migrations/
│   ├── config/
│   │   ├── mappings/
│   │   └── connectors/
│   ├── data/
│   │   ├── raw/
│   │   ├── demo/
│   │   └── quarantine/
│   ├── scripts/
│   │   ├── seed_demo.py
│   │   ├── ingest_demo.py
│   │   └── verify_layer1.py
│   ├── tests/
│   │   ├── unit/
│   │   ├── integration/
│   │   ├── contract/
│   │   └── e2e/
│   ├── docker/
│   │   ├── docker-compose.yml
│   │   ├── Dockerfile
│   │   ├── .env.example
│   │   ├── pyproject.toml
│   │   ├── README.md
│   │   └── Makefile
```

If the existing repository already has a different but coherent structure, adapt rather than duplicate. However, preserve the architectural separation: connector → raw → normalize → validate → persist → API.

12. DEMO DATA REQUIREMENTS

The system must be demonstrable without requiring a live third-party account.

Create a deterministic synthetic enterprise dataset

- At least 50 customers.
- At least 20 employees.
- At least 40 deals with stages, amounts, owners, and dates.
- At least 20 projects.
- At least 75 support tickets, including repeated tickets for selected customers.
- At least 10 documents or document-like records with useful text metadata.
- Include realistic cross-entity references.
- Include deliberate quality issues in a separate test fixture: missing email, invalid date, unknown customer reference, duplicate row, malformed amount, unknown status.
- Use fixed seed/random state so the dataset is reproducible.

Important scenario for later layers

Include a customer that has multiple support tickets in a short period and an associated active deal. Layer 1 does not calculate churn or make recommendations, but it must preserve the raw facts and relationships needed for later layers to detect that scenario.

Sample CSV mapping configuration

```
source: csv_demo
entity: customers

columns:
  customer_id: source_id
  customer_name: name
  email_address: email
  customer_segment: segment
  industry_name: industry
  account_owner_id: owner_source_id
  status: status
  created_date: created_at
```

13. DATA QUALITY ENGINE

Implement explicit quality dimensions. At minimum measure:

Dimension	Example metric
Completeness	% required fields populated.
Validity	% records passing type/domain validation.
Uniqueness	Duplicate source identities detected.
Consistency	Foreign keys resolve to known canonical entities.
Timeliness	Source update time vs ingestion time where available.
Traceability	% canonical records with complete provenance.

Severity model

- **ERROR:** record cannot safely enter canonical store.
- **WARNING:** record is usable but has a known quality limitation.
- **INFO:** operational event.

Quality report

Every ingestion run must produce counts for fetched, raw-persisted, normalized, inserted, updated, unchanged, rejected, warnings, and failed batches. These numbers must reconcile logically. Do not show invented percentages.

Quarantine

Rejected records must be recoverable for debugging. Store source context, row/entity identity, validation error code, human-readable reason, and the raw record or a safe reference to it.

14. SECURITY AND SAFETY

- Use environment variables for API keys, database credentials, and source credentials.
- Provide .env.example with placeholders only.
- Never commit real secrets, tokens, cookies, or exported enterprise data.
- Use read-only credentials for source systems whenever the source supports permission scoping.
- Set outbound request timeouts.
- Restrict connector HTTP methods to GET for REST/GraphQL prototype adapters.
- Validate URLs/configuration to reduce accidental requests to unintended hosts.
- Log identifiers and operational metadata, not secrets or full sensitive payloads.
- Design for tenant/source isolation even if the prototype uses one demo organization.
- Use parameterized SQL/ORM queries.
- Validate file types and size limits for uploaded/imported files.
- Do not execute formulas/macros from Excel files.
- Do not allow imported source content to become executable instructions.

Human governance boundary

Layer 1 only observes and normalizes data. It must never autonomously send emails, modify CRM records, create contracts, change budgets, or perform external business actions. That boundary is consistent with the overall AI CEO design, where external actions require human sign-off.

15. TESTING REQUIREMENTS

The implementation is incomplete until the following test categories exist and pass.

Test layer	Minimum coverage
Unit	Mapping, type coercion, identifier generation, hashing, validation rules, pagination helpers.
Connector contract	Every connector obeys the same interface and never uses write methods.
Database integration	Migrations, constraints, upserts, foreign keys, run tracking.
API	Health, source listing, ingestion, runs, errors, entity queries, structured failures.
End-to-end	Demo source → ingestion → PostgreSQL → API query.
Idempotency	Same source ingested twice without duplicate canonical identities.
Failure recovery	Malformed row, network error, DB failure, partial batch.
Security	Secrets not returned/logged; invalid configuration rejected.
Regression	All tests pass from a clean environment.

Minimum acceptance scenarios

- Fresh Docker startup succeeds.
- Database migrations succeed from empty database.
- Demo seed succeeds.
- Full CSV ingestion succeeds.
- Second identical CSV ingestion produces no duplicate source identities.
- A bad fixture produces quarantined errors without corrupting valid records.
- API returns canonical customer/deal/ticket records with provenance.
- Odoom/mock connector can be health-checked and queried through the same abstraction.
- Test suite exits successfully.

16. OBSERVABILITY AND OPERATIONS

Use structured logs. Every ingestion run should have a correlation/run ID that appears in logs and database records.

Log events

- run_started / run_finished
- connector_health_check
- batch_fetched
- record_normalized
- record_rejected
- batch_committed
- retry_scheduled
- run_failed

Metrics

- records_fetched_total
- records_inserted_total
- records_updated_total
- records_unchanged_total
- records_rejected_total
- ingestion_duration_seconds
- connector_request_failures_total
- validation_errors_total

Health endpoints

Liveness should answer whether the API process is alive. Readiness should verify required dependencies such as PostgreSQL. Connector health should be separate because a healthy application does not imply that every external source is reachable.

17. LOCAL DEPLOYMENT AND DEVELOPER EXPERIENCE

The project must be runnable with Docker Compose and also support direct Python development.

Required services

Service	Purpose
api	FastAPI application.
postgres	Canonical + operational persistence.
optional mock-source	Deterministic source service if used to demonstrate REST ingestion.

Required commands

```
make install
make migrate
make seed
make ingest-demo
make test
make lint
make format
make verify-layer1
docker compose up --build
```

The exact commands may be adapted to the repository, but there must be a small, documented command surface that a reviewer can learn quickly.

README must include

- Architecture diagram in text/Markdown.
- Prerequisites.
- Environment setup.
- Database migration.
- Demo dataset creation.
- Ingestion commands.
- API usage examples.
- Troubleshooting.
- Test commands.
- Layer 1 acceptance checklist.

18. IMPLEMENTATION PLAN FOR THE CODING AI

Phase	Work	Exit criterion
A	Repository audit + architecture decision	No unexplained duplicate infrastructure; implementation plan committed.
B	Canonical schemas + migrations	Empty DB can migrate; models validate.
C	Persistence + provenance	CRUD/upsert and run tracking work.
D	Normalization + quality engine	Fixtures normalize deterministically and invalid rows are quarantined.
E	CSV/Excel connector	Demo files ingest end-to-end.
F	Odoo/demo connector	Read-only adapter works or deterministic local mock proves contract.
G	Generic REST connector	GET-only adapter passes connector contract tests.
H	Ingestion orchestrator	Full + incremental/idempotent behavior works.
I	FastAPI	All required routes documented and tested.
J	Observability + security	Structured logs, metrics, secret hygiene, health checks.
K	Docker + CI	Clean environment reproduces build and tests.
L	Final verification	All acceptance criteria pass with evidence.

Implementation discipline

- After each phase, run the relevant tests before moving forward.
- Do not create placeholder functions that are silently accepted as complete.
- If a dependency is optional, isolate it and provide a fallback demo path.
- Prefer boring, maintainable infrastructure over unnecessary complexity.
- Use type hints, linting, formatting, docstrings for public interfaces, and meaningful error codes.

19. LAYER 2 HANDOFF CONTRACT

Layer 1 must make later layers easier without implementing them.

Downstream consumers should be able to request canonical entities using stable contracts such as:

```
GET /api/v1/entities/customers
GET /api/v1/entities/deals
GET /api/v1/entities/support_tickets
GET /api/v1/entities/employees
GET /api/v1/entities/projects
GET /api/v1/entities/documents
```

The Layer 2 Knowledge Graph will eventually map entities such as employees, customers, deals, projects, and tickets into relationships. The Layer 1 contract therefore must preserve stable source identity and canonical identity. The RAG layer will eventually consume document content and metadata, so document records should retain text plus provenance. Forecasting and agentic layers will eventually need timestamps, amounts, statuses, ownership, and event history.

Do not prematurely design Layer 1 around a specific graph schema or LLM framework. The canonical relational model is the boundary contract.

Future event interface

Optionally define an internal event model such as RecordUpserted, RecordRejected, and IngestionRunCompleted. It should be disabled or local-only in Layer 1 unless needed. The goal is to make future workflow/event integration possible without coupling the current implementation to a message broker.

20. FINAL FULL-PERFORMANCE ACCEPTANCE TEST

The coding AI must execute this scenario after implementation and report the actual observed results.

Scenario

- A. Start from a clean database.
- B. Start the application using Docker Compose.
- C. Verify API readiness.
- D. Seed deterministic enterprise demo data.
- E. Run a full CSV ingestion for customers, employees, deals, projects and support tickets.
- F. Confirm ingestion_run is SUCCESS or PARTIAL_SUCCESS with truthful counts.
- G. Query customers, deals and support tickets through the API.
- H. Verify source_system, source_entity, source_id, ingestion_run_id and timestamps are present.
- I. Run the exact same ingestion again.
- J. Confirm no duplicate canonical source identities exist.
- K. Run the deliberately malformed fixture.
- L. Confirm invalid rows are quarantined and valid rows remain usable.
- M. Run connector health checks.
- N. Run the entire unit + integration + contract + E2E test suite.
- O. Rebuild the stack from scratch and repeat the critical path.

Acceptance thresholds

Area	Pass condition
Build	Clean install/build succeeds.
Startup	All required services become healthy.
Migration	Empty PostgreSQL reaches expected schema.
Ingestion	Demo dataset reaches canonical tables without manual DB edits.
Idempotency	No duplicate source identities after repeated ingestion.
Traceability	Every accepted record has provenance.
Validation	Bad fixture produces structured errors/quarantine.
API	Canonical records are queryable with pagination.
Safety	No source write operations are executed.
Tests	All required automated tests pass.
Reproducibility	Clean rebuild reproduces the same demo behavior.
Documentation	README enables a new developer to run Layer 1.

The final response from the coding AI must contain

- What was implemented.
- Repository tree / major modules.
- Database schema summary.
- Connector list and supported source types.
- API endpoint list.
- Exact commands used to run and test it.
- Test results with counts.
- Known limitations, if any.
- Evidence that read-only behavior was preserved.
- Evidence that the end-to-end acceptance scenario passed.

21. ANTI-PATTERNS TO REJECT

Bad implementation	Required correction
Hard-coded CSV column names everywhere	Centralize source mapping configuration.
One giant ingestion function	Separate connector, normalization, validation, persistence, and orchestration.
LLM used to clean deterministic fields	Use deterministic code/rules; LLMs are not required in Layer 1.
Duplicate rows after rerun	Use source identity + idempotent upsert.
Missing source IDs	Require or deterministically derive a documented identity.
Errors only printed to console	Persist structured ingestion errors and run summaries.
Fake Odoos success response	Use a real read-only adapter or a clearly labeled deterministic mock implementing the same contract.
Secrets in .env committed to Git	Only commit .env.example placeholders.
No migrations	Use versioned migrations.
Future Neo4j schema embedded into ingestion code	Keep canonical relational contracts source-neutral.
Claiming production readiness	Call it a prototype and document limitations.

Quality bar

The implementation should look like the first subsystem of a serious platform, not like a classroom ETL script. It should be understandable by another developer, observable during a demo, safe against accidental source mutation, and extensible to later AI CEO layers.

22. FINAL DIRECTIVE TO THE CODING AI

Build Layer 1 completely. Do not merely tell me how to build it. Inspect the existing repository, implement the architecture, create the files, write the code, create migrations and demo fixtures, configure Docker, implement the connectors, expose the APIs, write the tests, run the tests, fix failures, and verify the final system.

When a design choice is ambiguous, prefer the simplest robust implementation that satisfies the requirements and document the decision. When a real external integration is unavailable, do not fabricate a live connection; provide a deterministic local adapter/mock with the same interface and make the boundary explicit.

Do not expand scope into the Knowledge Graph, RAG, forecasting, multi-agent, or human-action layers. Instead, make the canonical schema and APIs strong enough that those components can be attached later.

The final deliverable is a fully performing Layer 1 that can demonstrate:

- source connection → extraction → raw capture → normalization → validation → canonical PostgreSQL persistence → API retrieval
- provenance and ingestion-run tracking
- idempotent repeated ingestion
- structured data-quality failures
- read-only source behavior
- reproducible Docker deployment
- automated tests and final verification
- clean handoff to the future Enterprise Twin / Knowledge Graph layer

Do not mark the task complete until the final acceptance scenario has been executed successfully.

23. PROJECT SOURCE ALIGNMENT

This build prompt is grounded in the uploaded AI CEO project proposal and Review 1 materials. The proposal defines the Connector Layer as read-only API connectors and CSV ingestion that normalize data into a common schema, and its first implementation phase is the connector framework with read-only demo CRM/ERP ingestion and completed schema mapping.

The overall architecture remains: CRM/ERP/HRMS Systems → Connector Layer → Enterprise Twin + Knowledge Graph → Multi-Agent AI → Human Sign-off + Action. The prototype scope explicitly permits demo CRM/ERP data or public enterprise-style data and excludes production write access and fully autonomous execution.

Reference source: AI_CEO_Project_Proposal.pdf, especially the architecture, Layer Description, Scope, Technology Stack, and Methodology sections. The Review 1 presentation likewise identifies the Connector Framework as Weeks 1–4 and describes the connector deliverable as read-only demo CRM/ERP data with schema mapping complete.

Project source requirement	Layer 1 implementation interpretation
Read-only connectors	GET/read-only adapters with safety constraints and tests.
CSV/Excel ingestion	Deterministic file connector with configurable mappings.
Common schema	Canonical Pydantic + PostgreSQL entity contracts.
Demo CRM/ERP or public dataset	Deterministic demo data plus Odoo/demo-ready connector.
Connector framework, Weeks 1–4	Phased implementation ending in an end-to-end verified subsystem.
Future Enterprise Twin + KG	Stable downstream API/data contract, not implemented here.

End of document

Use Sections 2–22 as the implementation prompt. Keep this PDF alongside the repository as the Layer 1 engineering specification and acceptance contract.